Solution: House thief

There are `n` houses built in a line. A thief wants to steal the maximum possible money from these houses. The only restriction the thief has is that he can't steal from two consecutive houses, as that would alert the security system. How should the thief maximize his stealing?

Problem Statement

Given a number array representing the wealth of `n` houses, determine the maximum amount of money the thief can steal without alerting the security system.

Example 1:

```
Input: {2, 5, 1, 3, 6, 2, 4}
Output: 15
Explanation: The thief should steal from houses 5 + 6 + 4
```

Example 2:

```
Input: {2, 10, 14, 8, 1}
Output: 18
Explanation: The thief should steal from houses 10 + 8
```

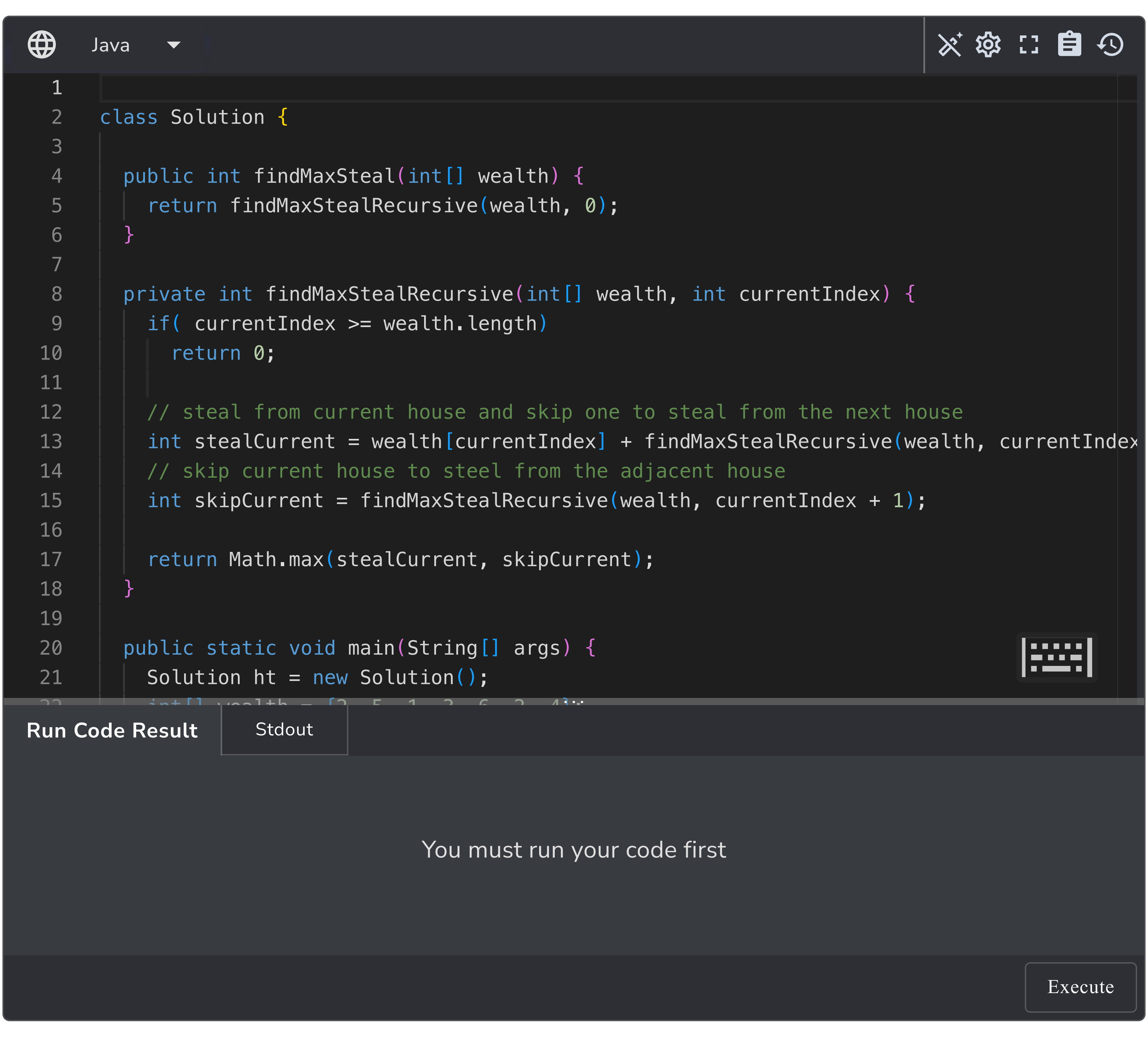
Let's first start with a recursive brute-force solution.

Basic Solution

For every house `i`, we have two options:

1. Steal from the current house (`i`), skip one and steal from (`i+2`).
2. Skip the current house (`i`), and steal from the adjacent house (`i+1`).

The thief should choose the one with the maximum amount from the above two options. So our algorithm will look like this:



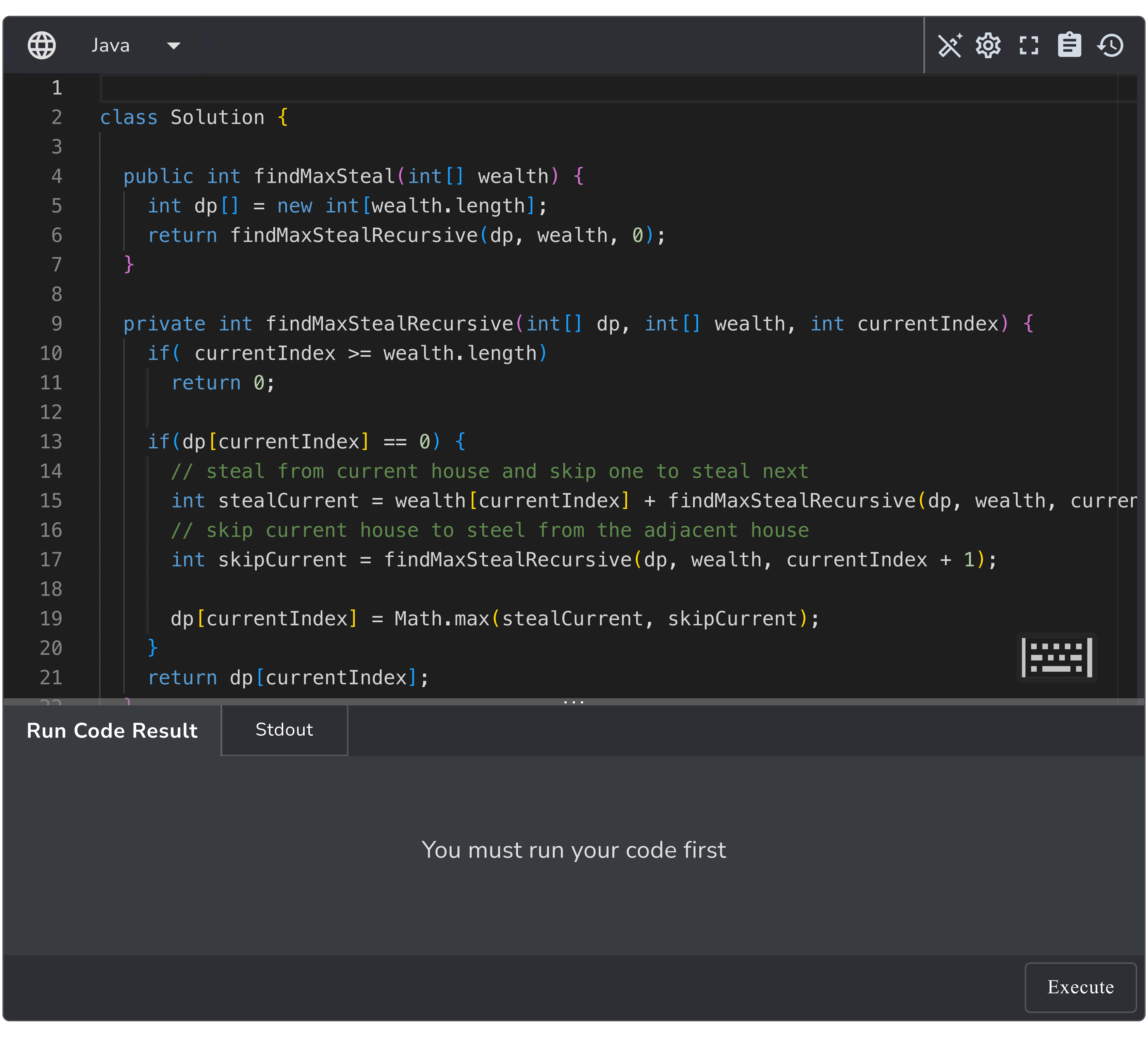
```java

class Solution {

  public int findMaxSteal(int[] wealth) {
    return findMaxStealRecursive(wealth, 0);
  }

  private int findMaxStealRecursive(int[] wealth, int currentIndex) {
    if( currentIndex >= wealth.length)
      return 0;

    // steal from current house and skip one to steal from the next house
    int stealCurrent = wealth[currentIndex] + findMaxStealRecursive(wealth, currentIndex
    // skip current house to steel from the adjacent house
    int skipCurrent = findMaxStealRecursive(wealth, currentIndex + 1);

    return Math.max(stealCurrent, skipCurrent);
  }

  public static void main(String[] args) {
    Solution ht = new Solution();
```

The time complexity of the above algorithm is exponential $O(2^n)$. The space complexity is $O(n)$ which is used to store the recursion stack.

Top-down Dynamic Programming with Memoization

To resolve overlapping subproblems, we can use an array to store the already solved subproblems.

Here is the code:



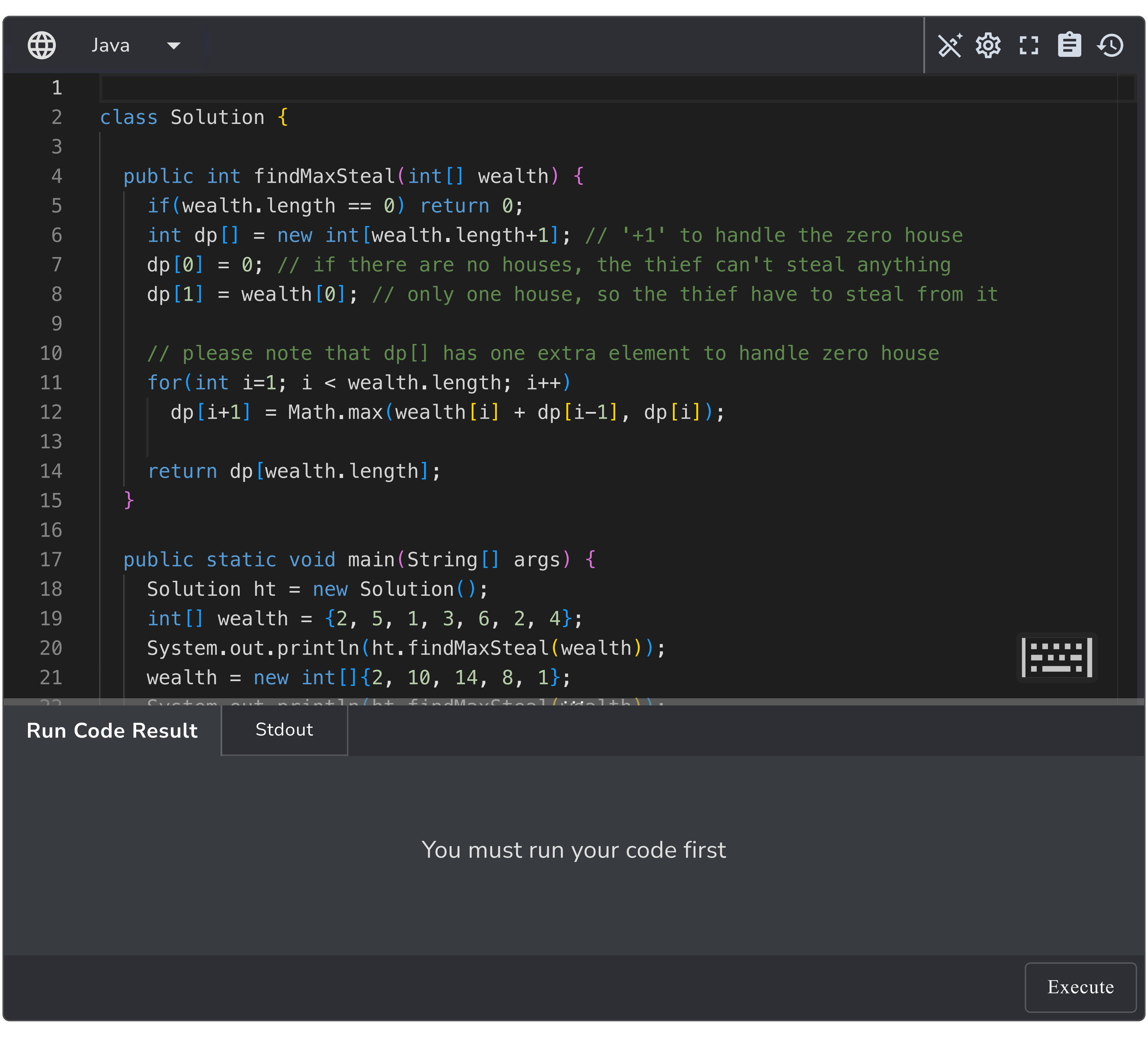
```java

class Solution {

  public int findMaxSteal(int[] wealth) {
    int dp[] = new int[wealth.length];
    return findMaxStealRecursive(dp, wealth, 0);
  }

  private int findMaxStealRecursive(int[] dp, int[] wealth, int currentIndex) {
    if( currentIndex >= wealth.length)
      return 0;

    if(dp[currentIndex] == 0) {
      // steal from current house and skip one to steal next
      int stealCurrent = wealth[currentIndex] + findMaxStealRecursive(dp, wealth, currer
      // skip current house to steel from the adjacent house
      int skipCurrent = findMaxStealRecursive(dp, wealth, currentIndex + 1);

      dp[currentIndex] = Math.max(stealCurrent, skipCurrent);
    }
    return dp[currentIndex];
```

Bottom-up Dynamic Programming

Let's try to populate our `dp[]` array from the above solution, working in a bottom-up fashion. As we saw in the above code, every `findMaxStealRecursive()` is the maximum of the two recursive calls; we can use this fact to populate our array.

Code

Here is the code for our bottom-up dynamic programming approach:

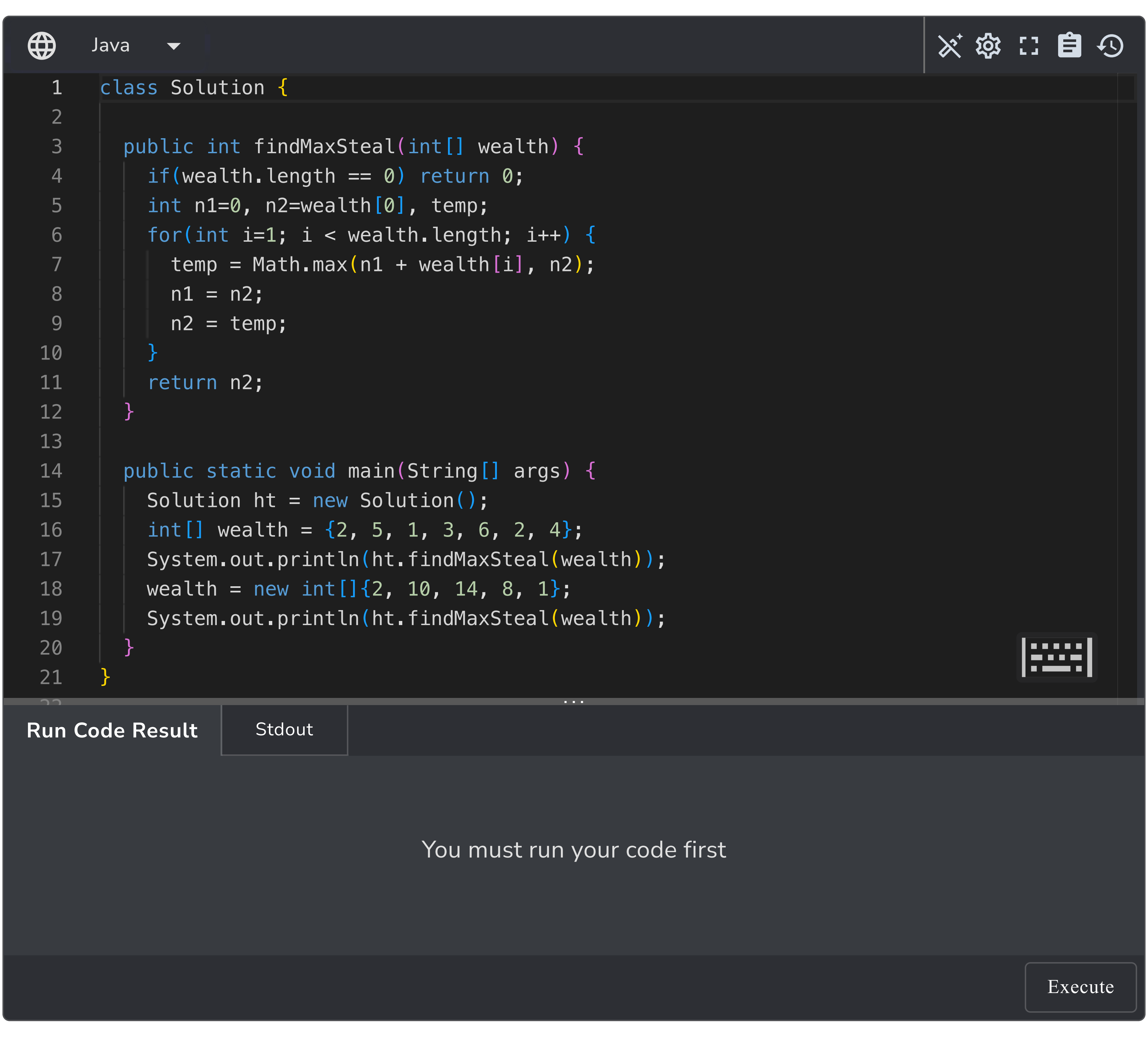


```java

class Solution {

  public int findMaxSteal(int[] wealth) {
    if(wealth.length == 0) return 0;
    int dp[] = new int[wealth.length+1]; // '+1' to handle the zero house
    dp[0] = 0; // if there are no houses, the thief can't steal anything
    dp[1] = wealth[0]; // only one house, so the thief have to steal from it

    // please note that dp[] has one extra element to handle zero house
    for(int i=1; i < wealth.length; i++)
      dp[i+1] = Math.max(wealth[i] + dp[i-1], dp[i]);

    return dp[wealth.length];
  }

  public static void main(String[] args) {
    Solution ht = new Solution();
    int[] wealth = {2, 5, 1, 3, 6, 2, 4};
    System.out.println(ht.findMaxSteal(wealth));
    wealth = new int[]{2, 10, 14, 8, 1};
```

The above solution has time and space complexity of $O(n)$.

Memory optimization

We can optimize the space used in our previous solution. We don’t need to store all the previous numbers up to `n`, as we only need two previous numbers to calculate the next number in the sequence. Let's use this fact to further improve our solution:



```java
class Solution {

  public int findMaxSteal(int[] wealth) {
    if(wealth.length == 0) return 0;
    int n1=0, n2=wealth[0], temp;
    for(int i=1; i < wealth.length; i++) {
      temp = Math.max(n1 + wealth[i], n2);
      n1 = n2;
      n2 = temp;
    }
    return n2;
  }

  public static void main(String[] args) {
    Solution ht = new Solution();
    int[] wealth = {2, 5, 1, 3, 6, 2, 4};
    System.out.println(ht.findMaxSteal(wealth));
    wealth = new int[]{2, 10, 14, 8, 1};
    System.out.println(ht.findMaxSteal(wealth));
  }
}
```

The above solution has a time complexity of $O(n)$ and a constant space complexity $O(1)$.

Fibonacci number pattern

We can clearly see that this problem follows the Fibonacci number pattern. The only difference is that every Fibonacci number is a sum of the two preceding numbers, whereas in this problem every number (total wealth) is the maximum of previous two numbers.

← Previous
House Thief

Next →
Longest Palindromic Subsequence

☐ Mark as Completed